

# Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Anonymous Authors

Index Terms—

## I. Preliminaries and Problem Statement

Native LGP provides a more integrated formulation of task and motion than traditional symbolic planning pipelines, making it attractive for robotic assembly. However, prior long-horizon formulations remain difficult to scale in complex stacking settings, where the solver may explore many symbolic alternatives before geometric infeasibility can effectively prune the search. As a result, directly solving long-horizon assemblies in native LGP often incurs high front-end search cost and limited practical scalability. To address this limitation, we introduce VLM-LGP, which inserts a visual-structural interface before native LGP so that the long-horizon task can be reorganized into a form that is more suitable for downstream solving.

We formulate the target structure as an object-support graph and seek a precedence-consistent decomposition into solver-facing batches whose scale remains manageable for downstream native LGP solving.

This formulation makes explicit that the central challenge is not whether native LGP can express the task, but how to expose only a solver-manageable portion of the long-horizon structure at each step. Mixed support patterns, such as bridge-like substructures, further complicate the symbolic-geometric search because local support changes can affect distant parts of the same full-horizon problem. These observations motivate a decomposition stage between support-graph generation and native LGP instantiation, so that structural grouping and precedence-respecting execution order can be resolved before downstream geometric solving.

## II. Method

VLM-LGP addresses long-horizon assembly through a staged pipeline that bridges visual task understanding and native LGP solving. At a high level, the framework is organized around two parallel streams. Task Decomposition, consisting of VLM Graph Generation, Two-Level Clustering, and Global Execution Sequence, converts the target assembly image into a solver-facing execution structure. In parallel, Scene Grounding, consisting of Reachability Screening and Manipulability-Aware Ordering, prepares scene-side feasibility information for the available object instances. The two streams are merged at Execution-Order Embedding and then passed to Selective LGP Solving

with Active Constraints. This organization preserves the expressive benefits of native LGP while avoiding a monolithic full-horizon formulation. Figure ?? summarizes the overall workflow, and the remainder of this section details each stage.

### A. VLM Graph Generation

We use the VLM to generate an object-support graph from the target assembly image, rather than asking it to predict precise object coordinates or a long executable program directly. This choice follows the capability boundary of current VLMs: they are comparatively reliable at inferring relational structure, support dependencies, and coarse left-right or above-below organization, but less stable when required to output exact geometry, long symbolic code, or mathematically precise layouts. We therefore let the VLM produce the structural representation that it is best suited to provide. Formally, the generated graph is  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , where each node  $v \in \mathcal{V}$  denotes one target object instance and each directed edge  $(u, v) \in \mathcal{E}$  indicates that object  $u$  must support object  $v$  in the final assembly. The graph is directed because support precedence is asymmetric, and optional local labels such as left/right placement hints are retained only as geometric side information rather than as the execution order itself.

Given this structural representation, we follow the general logic of graph clustering and partitioning to decompose the long-horizon task into solver-facing subproblems [?]. In our setting, however, the goal is not generic graph partition quality alone. The decomposition must also preserve support precedence, respect solver scale, and yield an executable global order for downstream native LGP solving. We therefore instantiate this idea as a two-level procedure: Branch Clustering first groups structurally related support chains into coarse branches, and Layer-Based Cutting then refines each branch into small executable batches. The resulting support graph serves as the central interface of the pipeline, connecting VLM-based structural parsing to decomposition and selective native LGP instantiation. Figure ?? shows representative support graphs parsed from target assembly images.

### B. Two-Level Clustering

Directly exposing the full support graph to native LGP remains undesirable in long-horizon assembly because the resulting symbolic-geometric search is still too coarse and too expensive to solve efficiently. We therefore introduce

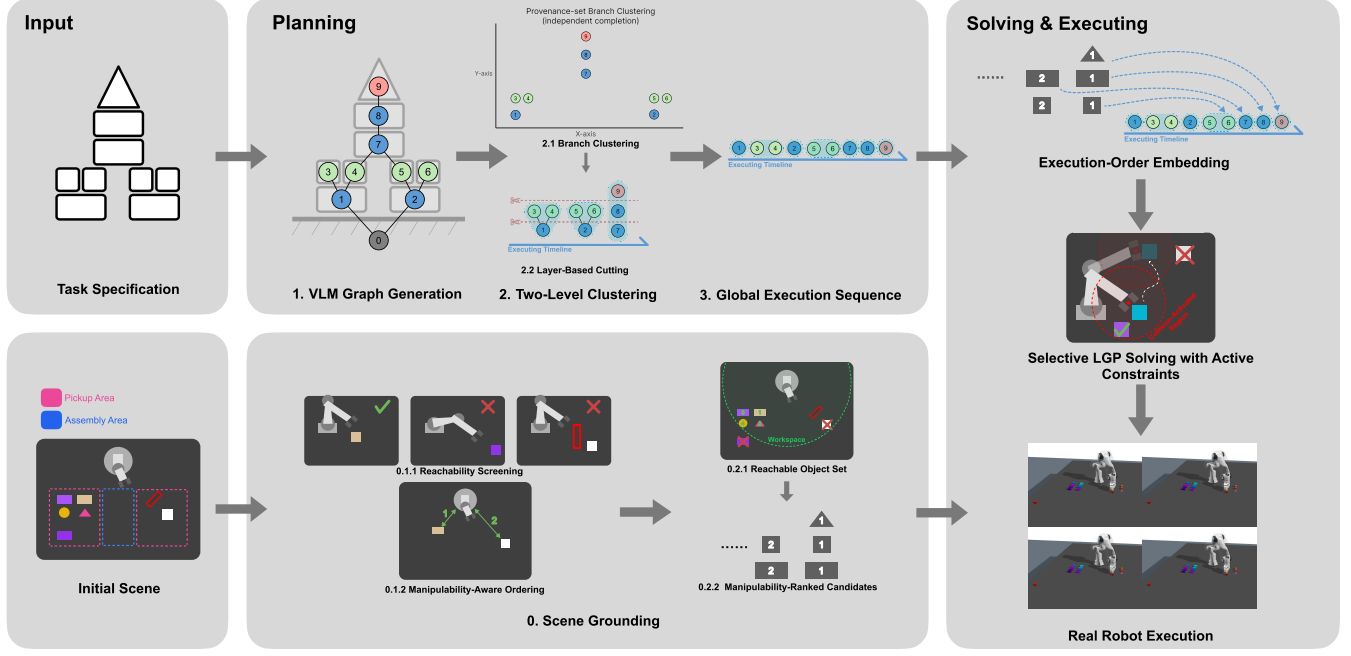


Fig. 1. The overall workflow of the proposed VLM-LGP framework.

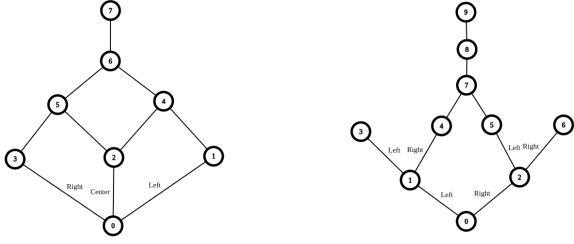


Fig. 2. Examples of object-support graphs generated by the VLM from target assembly images. Directed edges encode support relations that are later used for Branch Clustering and Layer-Based Cutting.

a graph-clustering stage that first aggregates the support graph into branch-level structures before any execution batches are generated. Among standard graph-clustering paradigms, the closest match to our setting is label propagation [?]: branch labels are seeded at root-supported nodes and then propagated upward along support edges so that nodes with the same support provenance fall into the same structural branch, while nodes that merge support from multiple roots become bridge branches. In our setting, the goal is not generic cluster compactness alone. The branch decomposition must also remain consistent with support precedence and preserve the coarse structural organization of the target assembly. On the representative example used in this paper, this first-stage branch clustering yields the three coarse groups 134, 256, and 789.

Branch-level grouping alone is still insufficient for downstream solving, because even after branch decomposition

a branch may remain too coarse for the practical solve-time budget and hardware limits of the current system. We therefore apply a second layer-based cutting stage that follows a simple execution logic inspired by human common sense: lower supporting layers are realized before upper ones, left and right side structures are expanded in a stable order, and objects on the same effective support level are grouped when they can be executed together. This second stage converts the coarse branches into solver-facing batches, for example  $134 \mapsto 1 \mid 34$ ,  $256 \mapsto 2 \mid 56$ , and  $789 \mapsto 7 \mid 8 \mid 9$ .

1) Global Execution Sequence: A final precedence-consistent merge over these local batch results produces the global execution sequence  $1 \mid 34 \mid 2 \mid 56 \mid 7 \mid 8 \mid 9$ .

### C. Scene Grounding

Scene Grounding prepares the initial scene for downstream solving by identifying which object instances remain executable and how they should be preferred when multiple feasible candidates are available. It contains two complementary stages: Reachability Screening removes infeasible objects, and Manipulability-Aware Ordering ranks the reachable remainder.

1) Reachability Screening: As the first component of Scene Grounding, reachability screening provides the scene-side feasibility counterpart to task decomposition. It does not determine the structural assembly order; instead, it prunes obviously infeasible objects before execution, so that later solver calls are not spent on targets that cannot be safely approached under the current scene geometry.

We implement this stage as a lightweight pick-waypoint feasibility test backed by FCL and libccd, allowing each candidate object to be screened through a shallow and

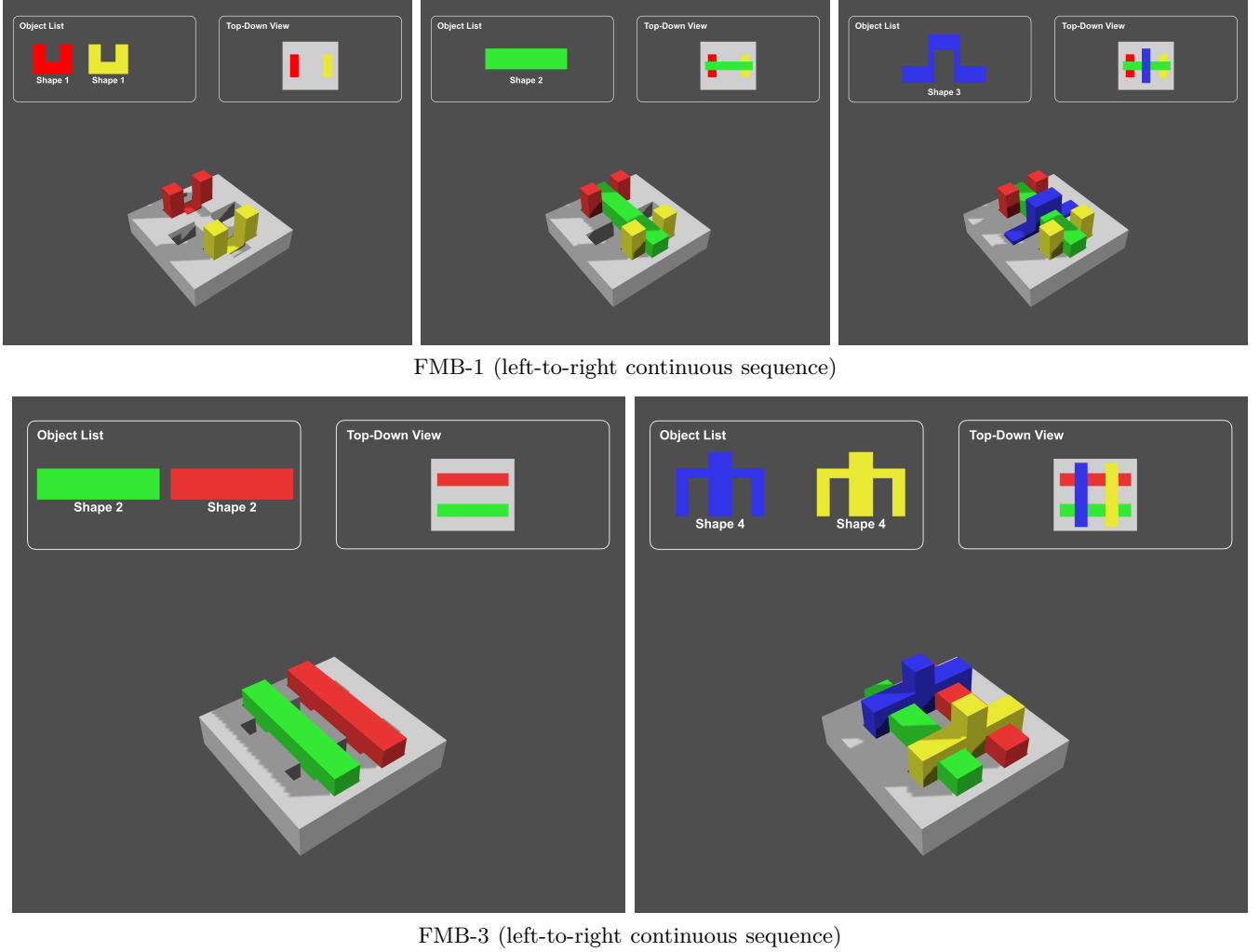


Fig. 3. FMB visual examples arranged as left-to-right continuous strips for each folder.

inexpensive geometric search before full planning. FCL performs broad-phase collision pairing, and libccd provides the fine collision and distance evaluation for the resulting candidate contacts. For each candidate object  $o_i$ , we instantiate a pick-waypoint subproblem  $\Pi_i^{\text{pick}}$  and declare the object reachable iff the solver returns a feasible solution:

$$\text{Reach}(o_i) = \mathbb{I}[\text{Feasible}(\Pi_i^{\text{pick}}(o_i))] \quad (1)$$

Objects that admit a feasible pick waypoint are retained for subsequent execution, whereas the remaining candidates are excluded as unreachable. This provides a shallow but geometry-consistent feasibility prior at low cost. Although the screening itself is lightweight, the same collision-evaluation logic is also compatible with the native LGP stack used later in the full pipeline.

2) **Manipulability-Aware Ordering:** Manipulability-Aware Ordering is applied only to the reachable object set returned by reachability screening. Its role is not to alter the structural precedence already produced by Task Decomposition, but to provide a scene-side preference over candidate instances when multiple reachable objects can satisfy the same structural request.

In the current implementation, this stage performs a static URDF-based kinematic analysis on each reachable candidate. For each object  $o_i$ , we define an approach target slightly above the object, solve a damped least-squares inverse-kinematics problem for that target, and evaluate the positional Yoshikawa manipulability score [?]

$$m(o_i) = \sqrt{\det(J_p(q_i)J_p(q_i)^T)}, \quad (2)$$

where  $J_p(q_i)$  denotes the position Jacobian at the corresponding solution configuration. The resulting scores are normalized to produce a manipulability-ranked candidate list, while low-confidence or IK-invalid cases are deferred to the tail of the corresponding object-type group. This stage therefore yields the manipulability-ranked candidates used later for scene-side binding.

#### D. Execution-Order Embedding

Execution-Order Embedding is the stage where the outputs of Task Decomposition and Scene Grounding are merged. Task Decomposition provides the global execution sequence together with solver-facing support batches, whereas Scene Grounding provides the reachable object

set and manipulability-ranked candidates for each object category. For each batch, the system embeds the structural order into the current scene by binding the batch-level object requests to concrete scene instances that survive reachability screening and are preferred by manipulability. The output of this stage is a grounded batch specification that can be directly consumed by the downstream LGP solver.

### E. Selective LGP Solving with Active Constraints

Native LGP couples symbolic task structure with continuous trajectory optimization through a shared representation of predicates, action semantics, and decision rules [?]. In principle, this abstraction can solve assembly tasks from symbolic goals alone. In long-horizon settings, however, directly exposing the full generic task domain still leads to excessive symbolic branching and unnecessary geometric backtracking.

Our solver therefore instantiates the native LGP abstraction selectively for each grounded execution batch. For the current batch, it compiles only the symbolic relations, object bindings, and decision rules required by the active support pattern. The solver thus receives a grounded symbolic-geometric subproblem tailored to the current batch rather than a monolithic domain covering all possible assembly alternatives.

We first keep the native LGP formulation in its original joint form [?]

$$\min_{\substack{x, a_{1:K}, \\ s_{1:K}, t_{1:K}}} \int_0^T c(x(t), \dot{x}(t), \ddot{x}(t)) dt + \psi(x(T)) \quad (3)$$

$$\text{s.t. } \forall k = 1:K \quad s_k = \text{succ}(a_k, s_{k-1}), \quad (4)$$

$$s_K \models g, \quad (5)$$

$$\forall_{k=1}^K h_{\text{switch}}(x(t_k) \mid a_k, s_{k-1}) = 0, \quad (6)$$

$$\forall_{k=1}^K g_{\text{switch}}(x(t_k) \mid a_k, s_{k-1}) \leq 0, \quad (7)$$

$$\forall t \in [0, T] \quad h_{\text{path}}(x(t), \dot{x}(t) \mid s_{k(t)}) = 0, \quad (8)$$

$$\forall t \in [0, T] \quad g_{\text{path}}(x(t), \dot{x}(t) \mid s_{k(t)}) \leq 0. \quad (9)$$

For each grounded batch, we then solve the corresponding KOMO subproblem in its original constrained sum-of-squares form [?]

$$\min_{x_{0:T}} \sum_{t=0}^T f_t(x_{t-k:t})^\top f_t(x_{t-k:t}) + \sum_{t,t'} k(t, t') x_t^\top x_{t'} \quad (10)$$

$$\text{s.t. } \forall t : g_t(x_{t-k:t}) \leq 0, \quad h_t(x_{t-k:t}) = 0. \quad (11)$$

The same waypoint-level solve also provides geometric cues for downstream motion optimization. Let  $x$  denote the full-motion trajectory optimized in the downstream stage, let  $d_{ij}(x)$  denote the minimum pairwise distance between collision proxy frames  $f_i$  and  $f_j$  along  $x$ , and let  $f_i^w$  denote the pose of frame  $f_i$  at waypoint  $w$ . The downstream trajectory is obtained by solving

$$x^* = \arg \min_x J(x), \quad (12)$$

Fig. 4. FMB VLM accuracy summary (from extracted FINAL\_JSON blocks).

subject to collision constraints instantiated only for the active set

$$c_{ij}(x) = d_{\text{safe}} - d_{ij}(x) \leq 0, \quad \forall (f_i, f_j) \in \mathcal{C}_{\text{active}}(w), \quad (13)$$

where

$$\mathcal{C}_{\text{active}}(w) = \{(f_i, f_j) \mid d(f_i^w, f_j^w) < r_{\text{active}}\}. \quad (14)$$

Here,  $J(x)$  denotes the downstream motion objective,  $d_{\text{safe}}$  is the prescribed safety margin, and  $r_{\text{active}}$  is the activation radius used to screen candidate collision pairs from waypoint geometry. Only pairs selected by  $\mathcal{C}_{\text{active}}(w)$  are instantiated as explicit collision constraints in the downstream full-motion solve. This active-constraint strategy reduces unnecessary collision checks and shortens full-motion solve time without altering the structural plan itself.

This restricted instantiation preserves the expressiveness of native LGP while concentrating the search on the structurally relevant portion of the task. It also provides a clean interface between graph-level decomposition and geometric solving: the decomposition module decides which support relations should be realized next, and native LGP resolves whether and how those relations can be executed under the current scene geometry.

### F. Real Robot Execution

After each subproblem is solved, the resulting trajectory is parsed and dispatched to the robot controller for execution on the physical platform. The execution layer remains intentionally thin: it reads the planned motion and gripper commands, executes them on the real robot, and updates the scene state for the next planning-solving cycle.

### References

- [1] M. Toussaint, “Logic-geometric programming for multi-step manipulation,” in Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI), 2015.
- [2] M. Toussaint, “Newton methods for k-order Markov optimization,” arXiv preprint arXiv:1407.0414, 2014.
- [3] G. Karypis and V. Kumar, “Multilevel k-way partitioning scheme for irregular graphs,” Journal of Parallel and Distributed Computing, vol. 48, no. 1, pp. 96–129, 1998.
- [4] U. N. Raghavan, R. Albert, and S. Kumara, “Near linear time algorithm to detect community structures in large-scale networks,” Physical Review E, vol. 76, no. 3, p. 036106, 2007.
- [5] T. Yoshikawa, “Manipulability of robotic mechanisms,” The International Journal of Robotics Research, vol. 4, no. 2, pp. 3–9, 1985.
- [6] J. Schoeler, F. T. Rehder, and M. Toussaint, “R-LGP: Reactive Logic-Geometric Programming,” arXiv preprint arXiv:2310.02791, 2023.
- [7] J. Doe et al., “Receding Horizon Hierarchical Logic-Geometric Programming,” arXiv preprint arXiv:2110.03420, 2021.
- [8] F. Lin et al., “Text2Motion: From Natural Language to Logic-Geometric Programming,” arXiv preprint arXiv:2303.12153, 2023.
- [9] General VLA Reference, “Vision-Language-Action Models for Robotics,” Placeholder, 2024.

- [10] H. Kopka and P. W. Daly, A Guide to L<sup>A</sup>T<sub>E</sub>X, 3rd ed., 1999.
- [11] M. Toussaint et al., “General VLA Reference for Task and Motion Planning,” arXiv preprint arXiv:2410.02193, 2024.
- [12] J. Doe et al., “Vision-Language Models for Robotic Assembly,” arXiv preprint arXiv:2407.09829, 2024.
- [13] Y. Zhu et al., “Fabrica: Dual-Arm Assembly of General Multi-Part Objects via Integrated Planning and Learning,” arXiv preprint arXiv:2506.05168, 2024.
- [14] D. McDermott et al., “PDDL-the planning domain definition language,” Technical Report, Yale Center for Computational Vision and Control, 1998.